Documento Explicativo: Uso de Estructuras de Datos

Proyecto: Sistema Policial de Gestión de Casos (Motor C++)

Sebastián Chisaba Cruz
Ariadna Valentina Ramírez Santeliz
Jerónimo Pascuas Barroso

24 de octubre de 2026

Resumen

El presente documento detalla la justificación teórica y el diseño técnico de implementación de cada una de las estructuras de datos empleadas en el desarrollo del **Sistema Policial de Gestión de Casos**. El proyecto separa la lógica de estructuras de datos en C++ (`Estructuras.h`) y su servidor de comunicación local (`servidor.cpp`) de la capa de visualización gráfica en HTML, CSS y JavaScript. Asimismo, se analiza la complejidad algorítmica temporal, la gestión automática de memoria dinámica y la declaración de uso de Inteligencia Artificial (45 %).

Índice

1. Resumen Arquitectónico del Sistema **2**

2. Análisis Detallado por Estructura de Datos **2**
2.1. Cola con Prioridad (`ColaPrioridadCasos` y `NodoCaso`) 2
2.2. Pila (`PilaEvidencias` y `NodoPila`) 3
2.3. Cola Circular (`ColaCircularCoartadas`) 4
2.4. Registros (`struct Sospechoso` y `struct Detective`) 4

3. Comunicación Modular C++ e Interfaz Gráfica Web **5**

4. Gestión de Memoria y Prevención de Fugas **5**

5. Declaración de Uso de Inteligencia Artificial **5**

1. Resumen Arquitectónico del Sistema

El sistema modela el flujo operativo de una central de detectives mediante una arquitectura desacoplada: todas las estructuras de datos residen y operan de forma nativa en memoria RAM dentro de un motor en C++ (`Estructuras.h` y `servidor.cpp`), el cual expone el estado de los nodos y registros en tiempo real hacia una interfaz gráfica web intuitiva (`index.html`, `styles.css` y `app.js`). Para resolver cada requerimiento con eficiencia computacional, se integraron cuatro estructuras de datos fundamentales[cite: 1, 2]:

Estructura de Datos	Clase / Entidad en C++	Memoria	Complejidad Principal
Cola con Prioridad	`ColaPrioridadCasos` `NodoCaso`	Dinámica	Inserción: $O(n)$ Atención y Cierre: $O(1)$
Pila	`PilaEvidencias` `NodoPila`	Dinámica	Apilar (Push): $O(1)$ Desapilar (Pop): $O(1)$
Cola Circular	`ColaCircularCoartadas`	Estática	Rotación circular: $O(1)$
Registros	`Sospechoso` `Detective`	Estática	Acceso directo: $O(1)$

Tabla 1: Resumen de estructuras de datos y complejidades algorítmicas del proyecto.

2. Análisis Detallado por Estructura de Datos

2.1. Cola con Prioridad (`ColaPrioridadCasos` y `NodoCaso`)

¿Por qué se utilizó?

En una central de emergencias policiales, los expedientes criminales no pueden atenderse bajo un esquema estrictamente secuencial de llegada, dado que la urgencia varía según la naturaleza del delito[cite: 1, 2]. Un caso de Prioridad 1 (Alta, ej. *Homicidio*) requiere atención inmediata por encima de un caso de Prioridad 2 (Media) o Prioridad 3 (Baja)[cite: 1, 2]. Adicionalmente, dado que la interfaz dispara alertas policiales automáticas cada 60 segundos, el número total de expedientes activos es indeterminado, haciendo indispensable una estructura dinámica que crezca bajo demanda sin desbordarse[cite: 1, 2].

¿Cómo se implementó?

En `Estructuras.h`, se implementó mediante una lista simplemente enlazada ordenada a través del atributo `prioridad` de la clase `NodoCaso`[cite: 1, 2]:

- **Inserción ordenada (`insertarConPrioridad`):** Al instanciar un nuevo nodo en el *Heap* con `new NodoCaso(...)`, se evalúa si la cola está vacía o si la prioridad del nuevo expediente es estrictamente mayor en urgencia (valor entero menor, p. ej., $1 < 2$) que el nodo en la `cabeza`[cite: 1, 2]. De cumplirse, el nuevo nodo se convierte en la nueva `cabeza` en tiempo $O(1)$[cite: 1, 2].
- **Estabilidad FIFO en empates:** Si el nuevo caso tiene menor o igual urgencia que la cabeza, un puntero auxiliar (`actual`) recorre la estructura bajo la condición `actual->sig->prioridad <= prio`[cite: 1, 2]. El uso del operador `<=` garantiza que, ante casos de igual prioridad, el recién llegado se ubique detrás de los que ya estaban esperando, preservando el orden cronológico justo[cite: 1, 2].
- **Desencolado (`eliminarPrimero`):** Como la estructura se mantiene ordenada desde la inserción, el caso más urgente siempre reside en `cabeza`[cite: 2]. Al emitirse la orden de arresto desde la interfaz web, se elimina el primer nodo en tiempo constante $O(1)$ y se libera su memoria mediante `delete`[cite: 2].

2.2. Pila (`PilaEvidencias` y `NodoPila`)

¿Por qué se utilizó?

Durante la investigación en el panel de la Sala de Evidencias, las pistas se recolectan progresivamente y el detective necesita visualizar en primera instancia el hallazgo más reciente en la parte superior de la columna central[cite: 2]. La filosofia **LIFO** (*Last In, First Out*) representa de manera exacta el apilamiento físico de reportes forenses[cite: 2]. Al implementarse de forma dinámica y anidada dentro de cada `NodoCaso`, cada expediente mantiene su propio historial aislado y solo reserva memoria por la cantidad exacta de evidencias procesadas (entre 0 y 4)[cite: 2].

¿Cómo se implementó?

Cada objeto `NodoCaso` contiene como atributo un objeto `PilaEvidencias`, gobernado por el puntero `NodoPila *cabeza`[cite: 2]:

- **Operación Push (`apilar`):** Cuando el usuario presiona el botón de nueva evidencia en el navegador, el servidor invoca `solicitarNuevaEvidencia()`, el cual descubre el siguiente rasgo del culpable y llama a `apilar(nuevaPista)`[cite: 2]. Se crea un `NodoPila` en memoria dinámica, su puntero `sig` apunta a la `cabeza` actual y `cabeza` pasa a ser el nuevo nodo en tiempo $O(1)$[cite: 2].
- **Operación Pop (`desapilar`) y Destructor:** El método `desapilar()` desvincula el nodo superior y ejecuta `delete temp`[cite: 2]. Cuando un caso se cierra en la cola de prioridad y se ejecuta `delete aux` sobre el `NodoCaso`, se invoca automáticamente el destructor `~PilaEvidencias()`, el cual vacía toda la pila nodo por nodo evitando fugas de memoria (*memory leaks*)[cite: 2].

2.3. Cola Circular (`ColaCircularCoartadas`)

¿Por qué se utilizó?

Cada vez que se genera un nuevo caso criminal, el sistema debe asignar una coartada declarada a cada uno de los 5 sospechosos[cite: 2]. Dado que el repertorio de declaraciones consta de un catálogo fijo de 10 coartadas reutilizables entre distintos casos, usar memoria dinámica implicaría reservar y destruir nodos innecesariamente[cite: 2]. Una **Cola Circular** permite reciclar el mismo bloque contiguo de memoria de forma perpetua en tiempo constante $O(1)$[cite: 2].

¿Cómo se implementó?

La clase `ColaCircularCoartadas` encapsula un arreglo estático `string elementos[10]` junto con los índices enteros `cabeza`, `ultimo` y `tamano`[cite: 2]:

- **Encolado inicial:** El constructor precarga las 10 coartadas avanzando el índice final con aritmética modular: `ultimo = (ultimo + 1) % 10`[cite: 2].
- **Desencolado con rotación (`desencolarYRotar`):** Al requerirse una coartada para un sospechoso, se extrae el elemento en `elementos[cabeza]`, se avanza el frente mediante `cabeza = (cabeza + 1) % 10` e inmediatamente se vuelve a insertar dicho texto al final de la fila (`ultimo = (ultimo + 1) % 10`)[cite: 2]. De este modo, las coartadas rotan en un ciclo infinito sin agotarse[cite: 2].

2.4. Registros (`struct Sospechoso` y `struct Detective`)

¿Por qué se utilizó?

Tanto los sospechosos como los detectives poseen atributos heterogéneos pero una cardinalidad fija dentro de las reglas de negocio del sistema (15 sospechosos en el catálogo global, 5 sospechosos por caso y 4 detectives en la plantilla)[cite: 2]. El uso de registros (`struct`) acoplados en arreglos estáticos permite encapsular todos los atributos de una entidad en una única unidad lógica con localidad espacial en caché y acceso directo en tiempo $O(1)$[cite: 2].

¿Cómo se implementó?

```
struct Sospechoso {
  string nombre;
  string rasgos[4];
  string coartada;
  bool esCulpable;
};

struct Detective {
  string nombre;
  bool disponible;
```

```
  int casosResueltos;
};
```

- **Gestión de Sospechosos:** En el constructor de `NodoCaso`, se selecciona aleatoriamente a 5 sospechosos distintos del catálogo global verificando mediante una tabla de marcas booleanas (`bool yaElegido[15]`) que ningún perfil se repita[cite: 2], copiando cada estructura directamente hacia el expediente y asignándole su coartada desde la cola circular.
- **Gestión de Detectives:** Se instancia el arreglo `Detective detectives[4]` en `servidor.cpp`, permitiendo que al hacer clic sobre un agente en la interfaz web se valide su estado booleano `disponible` y se asigne al caso prioritario en tiempo constante $O(1)$.

3. Comunicación Modular C++ e Interfaz Gráfica Web

Para dotar al proyecto de una interfaz gráfica moderna manteniendo el motor de estructuras de datos en C++, el sistema se dividió en dos capas:

1. **Capa Backend en C++ (`Estructuras.h` y `servidor.cpp`):** Ejecuta un socket TCP local en el puerto `8080` que procesa las rutas `/asignar`, `/evidencia`, `/arrestar` y `/alerta`, operando directamente sobre los punteros y arreglos en C++ y respondiendo un objeto JSON con el estado actualizado.
2. **Capa Frontend Web (`index.html`, `styles.css` y `app.js`):** Renderiza visualmente las tres columnas de trabajo (Cola con Prioridad y Detectives, Pila de Evidencias y Perfiles de Sospechosos) consumiendo el servidor C++ mediante peticiones asíncronas.

4. Gestión de Memoria y Prevención de Fugas

El proyecto garantiza una limpieza total y automática de la memoria dinámica (*Heap*) mediante destructores en dos niveles[cite: 2]:

1. **Al cerrar un caso individual:** El método `eliminarPrimero()` ejecuta `delete aux` sobre el `NodoCaso` resuelto, lo que dispara en cascada el destructor `~PilaEvidencias()`, liberando todos los nodos `NodoPila` asociados a ese expediente[cite: 2].
2. **Al finalizar el servidor:** El destructor `~ColaPrioridadCasos()` itera sobre todos los casos restantes invocando `eliminarPrimero()` hasta vaciar la cola por completo[cite: 2].

5. Declaración de Uso de Inteligencia Artificial

Por transparencia académica, los autores declaran un porcentaje estimado del **45 % de asistencia de herramientas de Inteligencia Artificial** durante el desarrollo del proyecto, distribuido específicamente en las siguientes tareas de apoyo:

- **Documentación e Interfaz Gráfica Web:** Estructuración de comentarios bajo el estándar Doxygen, maquetación visual en HTML/CSS/JS, conector HTTP en `servidor.cpp` y formato del documento en LaTeX.
- **Refactorización y pulido de clases:** Unificación de registros (`struct Sospechoso` y `struct Detective`), simplificación del algoritmo de selección aleatoria sin reemplazo y verificación de destructores para prevención de fugas de memoria.

La lógica central del juego, el diseño del flujo de casos criminales, la selección de estructuras de datos y las reglas de negocio del sistema policial fueron definidos y validados por los autores del proyecto.